

# FULLY DETAILED 9-MONTH (36-WEEK) WEB DEVELOPMENT PLAN

Starting: 15 December

Each week includes learning goals, tasks, exercises, mini-projects, resources, Google queries & LinkedIn post.

---

## MONTH 1 — WEB & UI FOUNDATIONS (Weeks 1–4)

---

### WEEK 1 — How the Web Works + Git

#### Learning Outcomes

- Understand DNS, hosting, servers, HTTPS, TCP/IP.
- Understand what happens when you type a URL.
- Install Git, configure SSH keys, push/pull to GitHub.

#### Tasks

- Draw a diagram of “How a webpage loads”.
- Set up Git + GitHub. Commit a simple file.
- Write a markdown README.
- Install VSCode & extensions (ESLint, Prettier, GitLens).

#### Exercises

- Explain HTTP request vs response in your own words.
- Clone a GitHub repo and push back changes.

#### Mini-Project

- “My First Web Repo” — Create a README with a diagram & explanation of how the web works.

#### What to Google

- “How DNS works simple explanation”
- “What happens when you type a URL and press enter GitHub version”
- “Git beginner workflow commands”

## Resources

- MDN — How the Web Works
- “How the Internet Works” (Vercel YouTube)
- GitHub Skills: Intro to Git

## LinkedIn Post

 Week 1 of my web dev journey!  
Learned how the web works under the hood and practiced Git for the first time.  
Excited for what's ahead!  
#WebDevelopment #LearningJourney

---

# WEEK 2 — Mastering Semantic HTML

## Learning Outcomes

- Understand semantic structure: header, nav, main, section, article, footer.
- Create forms with validation.
- Apply accessibility basics: ARIA labels, alt text, heading levels.

## Tasks

- Rebuild 2–3 pages from MDN HTML examples.
- Create a form with required validation + labels.
- Run Lighthouse accessibility audit.

## Exercises

- Convert a “div soup” layout to semantic HTML.
- Add ARIA attributes to a form.

## Mini-Project

- **Accessible Resume Page** — fully semantic, Lighthouse score > 90.

## What to Google

- “Semantic HTML benefits SEO accessibility”
- “ARIA labels examples”
- “HTML5 form validation attributes”

## Resources

- MDN HTML Guide
- WebAIM Accessibility Basics
- W3C Markup Intro

## LinkedIn Post

Week 2: Discovered that semantic HTML is basically “good manners” for the web. Accessibility matters!  
#HTML #Accessibility

---

# WEEK 3 — CSS Core + Layout Mastery

## Learning Outcomes

- Understand the box model deeply.
- Learn positioning: absolute, relative, fixed.
- Master Flexbox.
- Create responsive layouts.

## Tasks

- Recreate 5 common UI layouts using Flexbox.
- Resize browser to test responsive behavior.
- Rebuild a simple website home page layout from Dribbble.

## Exercises

- Use Flexbox to center items vertically & horizontally.
- Build mobile-first layout with media queries.

## Mini-Project

- **3-Page Responsive Website** (Home, About, Contact)

## What to Google

- “CSS specificity rules 2026”
- “Flexbox cheat sheet examples”
- “Responsive design mobile-first workflow”

## Resources

- CSS Tricks Flexbox Guide

- Kevin Powell YouTube
- MDN CSS Tutorials

### **LinkedIn Post**

CSS week! Flexbox finally clicked. Building responsive layouts feels like magic now.

#CSS #FrontendDevelopment

---

## **WEEK 4 — Advanced CSS + TailwindCSS**

### **Learning Outcomes**

- Build with CSS Grid.
- Use CSS variables, calc(), clamp().
- Learn Tailwind utility classes.
- Learn accessibility color contrast.

### **Tasks**

- Create a full landing page using only CSS Grid.
- Convert the same page to Tailwind.

### **Exercises**

- Create dark + light mode using CSS variables.
- Use clamp() for fluid typography.

### **Mini-Project**

- **Portfolio Landing Page (Tailwind Version)**

### **What to Google**

- “CSS Grid real-world examples”
- “Tailwind responsive utilities explained”
- “Fluid typography clamp() examples”

### **Resources**

- Tailwind Docs
- Grid Garden
- Web.dev Responsive Design

### **LinkedIn Post**

Finished Month 1! Tailwind + CSS Grid unlocked a whole new level of layout freedom.

#TailwindCSS #WebDesign

## MONTH 2 — JAVASCRIPT FUNDAMENTALS (Weeks 5–8)

---

### WEEK 5 — JavaScript Basics

#### Learning Outcomes

- Understand JS syntax: variables (let, const, var), types, operators.
- Understand DOM basics: selecting elements, changing content, event listeners.
- Learn console debugging and DevTools basics.

#### Tasks

- Create a simple HTML page and manipulate text with JS.
- Add click events and log results in the console.
- Practice `console.log`, `console.table`, and `debugger`.

#### Exercises

- Change the background color of a div on button click.
- Log each input value in a form on submit.

#### Mini-Project

- **Interactive To-Do List (Vanilla JS)** — add, remove, mark complete.

#### What to Google

- “JS DOM methods examples 2026”
- “Event listener best practices JS”
- “let vs const vs var”

#### Resources

- JavaScript.info
- MDN JavaScript Guide
- Codecademy JS Basics

## LinkedIn Post

Week 5: Officially started JavaScript! The DOM is a powerful playground. Built my first interactive to-do list. #javascript #webdev

---

## WEEK 6 — Functions & Arrays

### Learning Outcomes

- Write reusable functions.
- Understand scope, hoisting, closures.
- Master array methods: map, filter, reduce, forEach.

### Tasks

- Create functions for common operations: sum, max, string manipulation.
- Transform arrays using map/filter/reduce.

### Exercises

- Filter an array of numbers for even numbers.
- Map an array of objects to extract names.
- Reduce an array to get the sum of values.

### Mini-Project

- **Shopping Cart Total Calculator** — dynamically calculate totals with arrays and functions.

### What to Google

- “JS closure explained simply”
- “map vs forEach vs reduce examples”

### Resources

- Scrimba JS Bootcamp
- Fireship “Array Methods in 100 Seconds”
- MDN Array Reference

## LinkedIn Post

Week 6: Array methods blew my mind. map(), filter(), reduce() = cleaner, more powerful code. Built a dynamic shopping cart total calculator! #javascript #arrays

---

## WEEK 7 — Objects & JSON

### Learning Outcomes

- Work with JS objects and nested data.
- Learn JSON format, parsing, and stringifying.
- Practice passing objects into functions and arrays.

### Tasks

- Create objects representing users, products, or posts.
- Convert objects to JSON and back.

### Exercises

- Access nested object properties safely.
- Iterate over object keys/values.
- Fetch a sample JSON and display data dynamically.

### Mini-Project

- **Mini API Viewer** — fetch JSON from a placeholder API and render it.

### What to Google

- “JS objects vs classes 2026”
- “JSON parse vs stringify examples”

### Resources

- FreeCodeCamp JS Objects Course
- JSONPlaceholder API
- MDN Working with Objects

### LinkedIn Post

Week 7: Objects finally clicked. Fetched JSON from an API and displayed it dynamically. #javascript #API #webdev

---

## WEEK 8 — Async JavaScript

### Learning Outcomes

- Learn Promises, async/await, and fetch API.
- Handle errors using try/catch.
- Understand event loop basics.

### **Tasks**

- Fetch data from public APIs.
- Chain multiple async calls.
- Display data on the webpage with loading states.

### **Exercises**

- Use fetch() to get data from PokéAPI.
- Display loading spinner while waiting for data.
- Handle fetch errors gracefully.

### **Mini-Project**

- **Weather Dashboard** — fetch and display weather data from OpenWeatherMap API.

### **What to Google**

- “async await beginner examples”
- “fetch API tutorial 2026”
- “JS event loop explained”

### **Resources**

- MDN Promises Guide
- You Don't Know JS (Async)
- Traversy Media Async JS Crash Course

### **LinkedIn Post**

Week 8: Connected my first real API using fetch and async/await. Built a weather dashboard! Feeling powerful. #javascript #API #async

## **MONTH 3 — ADVANCED JAVASCRIPT + TYPESCRIPT (Weeks 9–12)**

---

### **WEEK 9 — ES6+ Features**

#### **Learning Outcomes**



- Learn modern JS syntax: arrow functions, template literals, destructuring.
- Understand spread/rest operators and default parameters.
- Use modules (import/export) for cleaner code structure.

### **Tasks**

- Rewrite older JS code using ES6 syntax.
- Split a script into multiple modules and import them.
- Experiment with default parameters in functions.

### **Exercises**

- Convert a for-loop into a map().
- Use destructuring to extract values from arrays and objects.
- Practice rest parameters to accept variable number of arguments.

### **Mini-Project**

- **Recipe Organizer** — modular JS app using ES6 imports/exports.

### **What to Google**

- “ES6 features cheat sheet 2026”
- “spread vs rest operator JS examples”
- “ES6 modules tutorial”

### **Resources**

- ES6 Handbook ([github.com/lukehoban/es6features](https://github.com/lukehoban/es6features))
- Babel REPL for testing features
- MDN ES6 Guides

### **LinkedIn Post**

Week 9: Modern JS features are a game-changer. Arrow functions, destructuring, and modules made my code cleaner and faster! #ES6 #javascript

---

## **WEEK 10 — TypeScript Fundamentals**

### **Learning Outcomes**

- Learn why TypeScript adds type safety.
- Understand basic types: string, number, boolean, arrays, tuples, enums.
- Use interfaces and type aliases for structured data.

## Tasks

- Convert small JS projects to TS.
- Define interfaces for objects.
- Use optional properties and readonly fields.

## Exercises

- Type an array of objects with an interface.
- Write functions with typed parameters and return types.
- Practice type assertions.

## Mini-Project

- **Typed To-Do List** — convert your Week 5 to-do list to TypeScript.

## What to Google

- “TypeScript interface vs type”
- “TypeScript beginner tutorial 2026”
- “why TypeScript is useful”

## Resources

- TypeScript Handbook
- Total TypeScript Course (free online)
- Traversy Media TS Crash Course

## LinkedIn Post

Week 10: TypeScript clicked today! Adding types reduces bugs and gives confidence when refactoring. Converted my to-do list project. #typescript #webdev

---

# WEEK 11 — Advanced TypeScript

## Learning Outcomes

- Learn generics, union & intersection types.
- Advanced type inference and type guards.
- Work with complex object types and arrays.

## Tasks

- Implement a generic function for reusable logic.

- Use union types for flexible parameters.
- Practice narrowing types with type guards.



## Exercises

- Create a generic API fetch function with type safety.
- Use union types for input values (string | number).
- Implement type guards for nested objects.



## Mini-Project

- **Typed API Viewer** — fetch JSON and strictly type all responses.



## What to Google

- “TypeScript generics examples 2026”
- “TS union vs intersection types”
- “type guards in TypeScript”



## Resources

- Effective TypeScript (book)
- TypeScript Deep Dive (online)
- Official TypeScript docs



## LinkedIn Post

Week 11: Generics and type guards blew my mind. TypeScript is finally feeling like a superpower! #typescript #webdev

---

# WEEK 12 — Testing with Jest



## Learning Outcomes

- Write unit tests in JS/TS using Jest.
- Understand test-driven development (TDD) basics.
- Learn to mock API calls.



## Tasks

- Write tests for functions created in Weeks 5–11.
- Mock a fetch API request.
- Check coverage reports and fix failing tests.



## Exercises

- Test array manipulation functions (map/filter/reduce).
- Write a test for a class or interface function.
- Use `beforeEach/afterEach` hooks.

### Mini-Project

- **Tested API Module** — create a small JS/TS module and write full unit tests.

### What to Google

- “Jest testing tutorial 2026”
- “unit testing JS examples”
- “TDD for beginners”

### Resources

- Jest Documentation
- Kent C. Dodds Testing JavaScript
- Fireship “Testing JS in 100 Seconds”

### LinkedIn Post

Week 12: Wrote my first unit tests and even mocked an API. Testing gives peace of mind and code confidence! #jest #testing #webdev

## MONTH 4 — REACT (Weeks 13–16)

---

### WEEK 13 — React Fundamentals

#### Learning Outcomes

- Understand the component-based architecture.
- Learn JSX syntax and rendering elements.
- Understand props for data passing.

#### Tasks

- Set up React project with Create React App / Vite.
- Create 3 simple components (Header, Card, Footer).
- Pass props from parent to child.

#### Exercises

- Render a list of items from an array using `.map()`.

- Add conditional rendering with ternary operators.

### Mini-Project

- **Simple User Profile Cards** — display users from an array of objects.

### What to Google

- “React components mental model”
- “JSX vs HTML differences”
- “props vs state in React”

### Resources

- React.dev Official Tutorial
- Scrimba React Course
- Wes Bos “React for Beginners”

### LinkedIn Post

Week 13: Started React — components make UI intuitive and reusable. Built my first user profile cards app! #react #frontend #webdev

---

## WEEK 14 — State & Effects

### Learning Outcomes

- Learn `useState` for dynamic state management.
- Learn `useEffect` for side effects like API calls.
- Understand dependency arrays and cleanup functions.

### Tasks

- Add interactivity with `useState` (like counters, toggles).
- Fetch data inside `useEffect` and render it.
- Clean up side effects properly.

### Exercises

- Create a toggle button with `useState`.
- Display fetched API data with a loading spinner.
- Console log state changes with `useEffect`.

### Mini-Project

- **Dynamic Weather Card** — fetch weather info and update on city input.

### What to Google

- “useEffect dependency array explained”
- “React re-render optimization”

### Resources

- React Official Hooks Docs
- David Ceddia Blog (Hooks deep dive)
- Scrimba React Hooks module

### LinkedIn Post

Week 14: useState + useEffect = magic! My pages finally feel alive. Built a dynamic weather card that fetches data live. #reacthooks #webdev

---

## WEEK 15 — Routing & Forms

### Learning Outcomes

- Learn React Router v6 for multi-page navigation.
- Create controlled forms with validation.
- Understand navigation via [Link](#) vs [NavLink](#).

### Tasks

- Create 2–3 pages with routes (Home, About, Contact).
- Implement a contact form with state handling.
- Navigate programmatically after form submission.

### Exercises

- Validate form fields dynamically.
- Implement a “Not Found” route for unknown URLs.
- Pass route parameters (params) to a component.

### Mini-Project

- **Portfolio Site** — 3 pages + contact form.

### What to Google

- “React Router v6 examples”

- “controlled vs uncontrolled form React”



## Resources

- React Router Docs
- Formik (form management)
- React Official Forms Docs



## LinkedIn Post

Week 15: Added multiple pages, navigation, and forms. My portfolio site feels real now! #reactrouter #frontend

---

# WEEK 16 — Data Fetching with React Query



## Learning Outcomes

- Learn React Query for server-state management.
- Understand caching, background fetching, and query invalidation.
- Manage loading, error, and success states.



## Tasks

- Install React Query and set up `QueryClientProvider`.
- Fetch data from public API and display results.
- Add caching and refetching logic.



## Exercises

- Handle API errors gracefully.
- Display cached data while fetching new data.
- Implement a refresh button to refetch queries.



## Mini-Project

- **Pokémon Search App** — fetch data from PokéAPI, search by name, show loading/error states.



## What to Google

- “React Query vs Redux”
- “React Query example fetching API”



## Resources

- TanStack React Query Docs
- Net Ninja React Query Crash Course
- PokéAPI Docs



### LinkedIn Post

Week 16: React Query makes API management a dream. Built a Pokémon search app with caching, loading states, and error handling. #reactquery #webdev



## MONTH 5 — NEXT.JS + AI TOOLS (Weeks 17–20)

---

### WEEK 17 — Next.js Basics



#### Learning Outcomes

- Understand Next.js folder structure: `app` vs `pages`.
- Learn server-side rendering (SSR) vs static site generation (SSG).
- Create dynamic routes.



#### Tasks

- Initialize a Next.js project with Vite/Next CLI.
- Build 3 pages (Home, About, Blog) with routing.
- Experiment with SSR (`getServerSideProps`) and SSG (`getStaticProps`).



#### Exercises

- Create a dynamic blog post route `[id].tsx`.
- Display static data with `getStaticProps`.
- Render server-side data with `getServerSideProps`.



#### Mini-Project

- **Simple Blog** — 3 posts, dynamic routing, SSR + SSG.



#### What to Google

- “Next.js App Router vs Pages 2026”
- “Next.js `getStaticProps` vs `getServerSideProps`”





## Resources

- Next.js Official Docs
- Fullstack Open Next.js section
- Vercel tutorials



## LinkedIn Post

Week 17: Started Next.js! Built a small blog using SSR & SSG. Full-stack development in one framework feels amazing. #nextjs #fullstack

---

# WEEK 18 — Server & Client Components



## Learning Outcomes

- Understand server components vs client components.
- Learn when to fetch data on server vs client.
- Optimize performance with server-side rendering.



## Tasks

- Convert existing components to server and client components.
- Fetch data server-side for static pages.
- Handle interactivity with client components.



## Exercises

- Add a client-only interactive button to a server component.
- Experiment with streaming server responses.
- Compare rendering performance metrics.



## Mini-Project

- **Portfolio Site with Mixed Components** — static server-rendered sections + interactive client widgets.



## What to Google

- “Server Components vs Client Components Next.js”
- “Next.js performance optimization tips”



## Resources

- Vercel Server Components deep dive
- t3 stack tutorials

- React Server Components official blog



## LinkedIn Post

Week 18: Server components are the future. Optimized my portfolio with server + client components for better performance. #nextjs #reactRSC

---

# WEEK 19 — Authentication



## Learning Outcomes

- Learn authentication flows (OAuth, JWT).
- Implement NextAuth for login/signup.
- Manage session securely.



## Tasks

- Configure NextAuth with GitHub/Google providers.
- Protect private routes.
- Handle session data in components.



## Exercises

- Create a protected dashboard page.
- Logout programmatically.
- Handle session expiration gracefully.



## Mini-Project

- **Protected Portfolio Dashboard** — login to see private project stats.



## What to Google

- “NextAuth 2026 tutorial”
- “OAuth login flow explained simply”



## Resources

- NextAuth.js Documentation
- Auth0 Tutorials
- Clerk.dev guides



## LinkedIn Post

Week 19: Implemented OAuth login with NextAuth. Authentication is no longer scary! #nextjs #auth

---

## WEEK 20 — AI APIs & Integrations

### Learning Outcomes

- Learn to connect to AI APIs (OpenAI, HuggingFace).
- Understand prompt design.
- Use AI for code generation, chatbots, or content features.

### Tasks

- Create a Next.js API route to query an AI model.
- Build a small chat interface.
- Handle errors and rate limits.

### Exercises

- Experiment with different prompts for code generation.
- Display AI responses in real-time.
- Secure API keys with environment variables.

### Mini-Project

- **AI Assistant Widget** — embedded in portfolio site, answers questions interactively.

### What to Google

- “OpenAI API tutorial Next.js”
- “HuggingFace Inference API example”
- “AI integration in web apps 2026”

### Resources

- OpenAI API Docs
- HuggingFace Inference API
- Google AI Studio tutorials

### LinkedIn Post

Week 20: Connected my first AI API! Built an interactive AI assistant widget on my portfolio. 🤖 #AI #webdev #nextjs

---



# MONTH 6 — NODE.JS & DATABASES (Weeks 21–24)

---

## WEEK 21 — Node.js & Express



### Learning Outcomes

- Set up Node.js environment, npm scripts.
- Create Express server with routing and middleware.
- Serve static files & JSON responses.



### Tasks

- Initialize Node.js project.
- Create Express endpoints (GET, POST).
- Log requests with middleware.



### Exercises

- Build CRUD endpoints for a sample dataset.
- Test endpoints with Postman.



### Mini-Project

- **REST API for Notes App** — create, read, update, delete notes.



### What to Google

- “Express routing examples 2026”
- “Node.js REST API tutorial”



### Resources

- Traversy Media Node.js Crash Course
- Express Docs
- Postman tutorials



### LinkedIn Post

Week 21: Built my first Node.js backend with Express. API endpoints returning data feel powerful! #nodejs #backend

---

## WEEK 22 — Databases (PostgreSQL)

### Learning Outcomes

- Learn SQL basics and database schema design.
- Connect Node.js app to PostgreSQL.
- Perform CRUD operations.

### Tasks

- Install PostgreSQL locally or use Supabase.
- Create tables for users/products.
- Connect Express routes to DB.

### Exercises

- Write SELECT, INSERT, UPDATE, DELETE queries.
- Join multiple tables.
- Use parameterized queries for security.

### Mini-Project

- **Notes App Backend** — persist notes in PostgreSQL database.

### What to Google

- “PostgreSQL beginner queries”
- “Node.js PostgreSQL tutorial”

### Resources

- SQLBolt interactive lessons
- Supabase tutorials
- PostgreSQL Official Docs

### LinkedIn Post

Week 22: Learned SQL and connected my Node.js app to a PostgreSQL database. Persistent storage is a game-changer! #SQL #backend

---

## WEEK 23 — Authentication & Security

### Learning Outcomes

- Learn JWT authentication, password hashing (bcrypt)

- Implement secure routes and OWASP best practices.
- Protect against common vulnerabilities.

### **Tasks**

- Add JWT login system.
- Hash passwords before saving.
- Protect sensitive endpoints.

### **Exercises**

- Create login/register routes.
- Test protected routes without token.
- Experiment with CORS and headers.

### **Mini-Project**

- **Secure Notes App** — users must login to see their notes.

### **What to Google**

- “JWT authentication Node.js tutorial”
- “bcrypt hashing Node.js”
- “OWASP top 10 security”

### **Resources**

- OWASP Cheat Sheets
- Net Ninja JWT Tutorial
- Bcrypt Documentation

### **LinkedIn Post**

Week 23: Implemented JWT authentication and hashed passwords. Security matters! #nodejs #security

---

## **WEEK 24 — ORM (Prisma)**

### **Learning Outcomes**

- Learn type-safe database access.
- Perform migrations and schema modeling.
- Integrate Prisma with Node.js/Next.js.

### **Tasks**

- Install Prisma, connect to PostgreSQL.
- Define schema for users/notes.
- Run migrations and test CRUD operations.

### Exercises

- Fetch data with Prisma client.
- Create relationships between tables.
- Practice complex queries.

### Mini-Project

- **Full-Stack Notes App with Prisma** — integrate frontend, backend, and database.

### What to Google

- “Prisma schema basics”
- “Prisma CRUD Next.js”

### Resources

- Prisma Docs
- PlanetScale + Prisma tutorials
- Fullstack Open Prisma section

### LinkedIn Post

Week 24: Prisma makes database operations type-safe and fast. Integrated frontend, backend, and database for my Notes App. #prisma #fullstack

## MONTH 7 — ADVANCED GIT & DEPLOYMENT (Weeks 25–27)

---

### WEEK 25 — Advanced Git

#### Learning Outcomes

- Learn branching strategies: GitFlow, feature branches.
- Understand merging, rebasing, and conflict resolution.
- Use pull requests and code reviews.

#### Tasks

- Create a GitFlow branch structure: develop, feature, release.
- Practice rebasing feature branches onto main.
- Resolve merge conflicts.

### Exercises

- Merge a feature branch with conflicts.
- Rebase a branch interactively.
- Push changes and create a pull request on GitHub.

### Mini-Project

- **Collaborative Git Simulation** — simulate a team workflow with multiple branches.

### What to Google

- “git rebase vs merge 2026”
- “GitFlow tutorial”
- “resolving Git conflicts example”

### Resources

- Pro Git Book
- Atlassian Git Tutorials
- “Oh Shit Git” cheatsheet

### LinkedIn Post

Week 25: Git finally makes sense! Branching, merging, and rebasing like a pro.  
Feeling like a real developer. #git #webdev

---

## WEEK 26 — Deployment

### Learning Outcomes

- Deploy front-end and full-stack apps.
- Understand Vercel, Netlify, Railway.
- Learn basic Docker for containerized deployments.

### Tasks

- Deploy Next.js project to Vercel.
- Deploy Node.js backend to Railway.
- Build a Docker container for your backend.



## Exercises

- Test live API endpoints after deployment.
- Set environment variables securely.
- Update deployed app and observe automatic rebuild.

## Mini-Project

- **Live Portfolio + Notes App** — deploy frontend and backend live.

## What to Google

- “how to deploy next.js app vercel 2026”
- “deploy node app railway tutorial”
- “docker beginner tutorial”

## Resources

- Vercel Docs
- Netlify Docs
- Railway Docs
- Docker Getting Started

## LinkedIn Post

Week 26: Deployed my projects live! Seeing my code in production is addictive.  
#deployment #fullstack

---

# WEEK 27 — Cloud Basics (AWS/GCP)

## Learning Outcomes

- Learn S3 for storage, compute basics (EC2/GCP Compute Engine).
- Understand cloud pricing and managed databases.
- Practice cloud-hosted DB connections.

## Tasks

- Upload assets to S3 / GCP Storage Bucket.
- Launch a basic compute instance.
- Connect managed PostgreSQL database to backend.

## Exercises

- Host static files in S3.

- Monitor usage and cost for small deployment.
- Test DB connection from cloud instance.

### **Mini-Project**

- **Portfolio + Cloud Storage Integration** — store uploaded images for portfolio projects.

### **What to Google**

- “AWS S3 beginner tutorial”
- “GCP compute vs AWS EC2”
- “cloud-hosted database setup PostgreSQL”

### **Resources**

- AWS Skill Builder
- Google Cloud Skills Boost
- FreeCodeCamp Cloud Basics

### **LinkedIn Post**

Week 27: Learned cloud fundamentals — S3, compute, and managed DBs. My apps are now truly cloud-ready. #cloud #webdev

---

## **MONTH 8 — CI/CD, Portfolio & Project Polish (Weeks 28–32)**

---

### **WEEK 28 — CI/CD (GitHub Actions)**

#### **Learning Outcomes**

- Automate builds, tests, and deployments.
- Learn to configure pipelines and workflows.
- Understand triggers for CI/CD.

#### **Tasks**

- Create a GitHub Actions workflow for testing and deployment.
- Run builds automatically on PR merge.
- Add notifications on workflow failure.

## Exercises

- Set up automated deployment to Vercel after merge.
- Add linting step to pipeline.
- Test CI/CD with a feature branch.

## Mini-Project

- **Automated Portfolio Deployment** — updates deploy automatically on GitHub push.

## What to Google

- “GitHub Actions deploy example 2026”
- “CI/CD pipeline tutorial”

## Resources

- GitHub Actions Docs
- Example workflows from GitHub
- FreeCodeCamp CI/CD tutorials

## LinkedIn Post

Week 28: Automated deployments and testing with GitHub Actions. Build → test  
→ deploy = magic! #cicd #automation

---

# WEEK 29 — Project Planning (Portfolio)

## Learning Outcomes

- Learn to scope MVP projects.
- Choose tech stack based on goals.
- Plan tasks and milestones.

## Tasks

- Define a portfolio project: features, pages, tech stack.
- Create a Trello or Notion board for tasks.
- Identify potential challenges.

## Exercises

- Break project into weekly milestones.
- Prioritize core features first.
- Estimate time for each task.

## Mini-Project

- **Portfolio Project Roadmap** — plan your final full-stack project end-to-end.

## What to Google

- “best full stack project ideas 2026”
- “how to scope MVP for web app”

## Resources

- Product planning guides
- Project templates
- Examples of developer portfolios

## LinkedIn Post

Week 29: Designed my final full-stack project roadmap. Excited to build a portfolio-ready app! #fullstack #planning

---

# WEEK 30 — Frontend Implementation

## Learning Outcomes

- Build UI with responsiveness and accessibility.
- Optimize performance with Lighthouse.
- Apply color theory, typography, spacing.

## Tasks

- Build pages with Tailwind/React components.
- Test responsiveness on multiple devices.
- Run Lighthouse performance & accessibility audits.

## Exercises

- Implement responsive navigation menu.
- Optimize images & assets
- Test contrast ratios for accessibility.

## Mini-Project

- **Portfolio Frontend Completed** — fully responsive and accessible.

## What to Google

- “Next.js layout best practices 2026”
- “web performance checklist”

## Resources

- Lighthouse
- Web.dev
- Figma design resources

## LinkedIn Post

Week 30: Frontend nearly done. UI is clean, responsive, and accessible.  
#frontend #webdev

---

# WEEK 31 — Backend Implementation

## Learning Outcomes

- Integrate backend with frontend.
- Handle secure API routes, caching, rate-limiting.
- Validate input server-side.

## Tasks

- Connect React/Next frontend to Node/Express backend.
- Secure endpoints with JWT.
- Cache API responses if needed.

## Exercises

- Test endpoints from frontend.
- Apply server-side validation.
- Implement simple rate-limiting middleware.

## Mini-Project

- **Full-Stack Portfolio App Backend** — secure and fully integrated.

## What to Google

- “API rate limiting Node.js”
- “Node.js caching strategies”

## Resources

- Redis tutorials
- Rate-limiting middleware guides
- Node.js security best practices

### **LinkedIn Post**

Week 31: Backend integrated! Authentication, database, APIs — all connected.  
#backenddeveloper

---

## **WEEK 32 — Deployment & Documentation**

### **Learning Outcomes**

- Launch full-stack project live.
- Monitor performance, errors, analytics.
- Write clear README and project docs.

### **Tasks**

- Deploy frontend & backend.
- Integrate Sentry/Analytics.
- Write detailed README with setup instructions.

### **Exercises**

- Test live app functionality.
- Document API endpoints.
- Monitor live errors & fix quickly.

### **Mini-Project**

- **Portfolio Project LIVE** — publicly accessible and documented.

### **What to Google**

- “post-launch website checklist”
- “how to write project documentation”

### **Resources**

- Sentry
- Google Analytics
- Documentation templates

### **LinkedIn Post**

Week 32: Project is LIVE! Public, documented, and monitored. This will be the centerpiece of my portfolio. #projectlaunch #fullstack

---

## MONTH 9 — JOB PREP & SPECIALIZATION (Weeks 33–36)

---

### WEEK 33 — Portfolio & Resume Polish

#### Learning Outcomes

- Finalize portfolio site.
- Create compelling case studies.
- Tailor resume for web dev roles.

#### Tasks

- Polish frontend, responsive layouts.
- Add project descriptions, tech stack, challenges.
- Update resume with portfolio link.

#### Exercises

- Add GitHub links to projects.
- Optimize images and performance.
- Check accessibility scores.

#### Mini-Project

- **Job-Ready Portfolio** — fully polished, responsive, accessible.

#### What to Google

- “developer portfolio examples 2026”
- “resume for junior developer samples”

#### Resources

- dev.to portfolio examples
- Templates for resumes
- UX/UI portfolio guides

## LinkedIn Post

Week 33: Polishing my portfolio and resume. It's starting to look really sharp.  
#portfolio #jobsearch

---

## WEEK 34 — Interview Prep (JS + DSA Lite)

### Learning Outcomes

- Review core JS concepts.
- Solve common array/string problems.
- Practice problem-solving in timed conditions.

### Tasks

- Solve 5–10 coding challenges per day (easy/medium).
- Review closures, scope, promises, and async.
- Prepare STAR answers for behavioral questions.

### Exercises

- LeetCode (Easy) array/string problems.
- Implement simple algorithm challenges.
- Mock coding interviews with a peer.

### Mini-Project

- **Interview Prep Notes** — document patterns and common questions.

### What to Google

- “Top JavaScript interview questions 2026”
- “array problems JavaScript solutions”

### Resources

- LeetCode (Easy)
- AlgoExpert
- Frontend interview prep guides

## LinkedIn Post

Week 34: Grinding JS interview questions. Feeling more confident with each practice session. #interviewprep #javascript



---

## WEEK 35 — System Design Basics

### Learning Outcomes

- Learn lightweight system design for web apps.
- Understand scalability, caching, and trade-offs.
- Design simple backend architectures.

### Tasks

- Design an architecture for a small-scale app.
- Identify bottlenecks and optimizations.
- Draw diagrams showing components, DB, APIs.

### Exercises

- Compare monolithic vs microservices for small app.
- Discuss pros/cons of caching strategies.
- Practice whiteboarding simple web systems.

### Mini-Project

- **System Design Diagrams** — for your portfolio project.

### What to Google

- “System design for frontend developers”
- “Scalable web app design 2026”

### Resources

- System design primers
- Real-world architecture case studies
- FreeCodeCamp design guides

### LinkedIn Post

Week 35: Learned lightweight system design. Great for interviews and planning real-world apps. #systemdesign #webdev

---

## WEEK 36 — Specialization & Next Steps

## Learning Outcomes

- Choose a specialization (Frontend, Full-Stack, DevOps, AI/ML).
- Plan job applications, interviews, or freelance work.
- Identify learning paths for continued growth.

## Tasks

- Choose main stack/role.
- Apply to 5–10 relevant job listings.
- Connect with mentors and communities.

## Exercises

- Build 1–2 mini-projects highlighting specialization
- Update portfolio to reflect specialization skills.
- Plan next 6 months of learning/upskilling.

## Mini-Project

- **Specialization Showcase Project** — portfolio highlight demonstrating chosen focus.

## What to Google

- “frontend developer job description 2026”
- “how to choose dev specialization”

## Resources

- Job boards: LinkedIn, Indeed, Angellist
- Mentorship programs
- Upskilling resources

## LinkedIn Post

Week 36: 9-month journey complete! Chose my specialization and feeling ready to enter the industry. Proud and excited for what's next. #webdeveloper #careerjourney